

```
In [9]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [10]: df = pd.read_csv('./data/ncr_ride_bookings.csv')
df.head()
```

Out[10]:

	Date	Time	Booking ID	Booking Status	Customer ID	Vehicle Type	Pickup Location	Dro Location
0	2024-03-23	12:29:38	"CNR5884300"	No Driver Found	"CID1982111"	eBike	Palam Vihar	Jhilm
1	2024-11-29	18:01:39	"CNR1326809"	Incomplete	"CID4604802"	Go Sedan	Shastri Nagar	Gurgaon Sector 5
2	2024-08-23	08:56:10	"CNR8494506"	Completed	"CID9202816"	Auto	Khandsa	Malviya Nagar
3	2024-10-21	17:17:25	"CNR8906825"	Completed	"CID2610914"	Premier Sedan	Central Secretariat	Inderlok
4	2024-09-16	22:08:00	"CNR1950162"	Completed	"CID9933542"	Bike	Ghitorni Village	Khanau Market

5 rows × 21 columns

```
In [11]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150000 entries, 0 to 149999
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Date                                  150000 non-null object
1   Time                                  150000 non-null object
2   Booking ID                           150000 non-null object
3   Booking Status                       150000 non-null object
4   Customer ID                          150000 non-null object
5   Vehicle Type                         150000 non-null object
6   Pickup Location                      150000 non-null object
7   Drop Location                       150000 non-null object
8   Avg VTAT                            139500 non-null float64
9   Avg CTAT                            102000 non-null float64
10  Cancelled Rides by Customer          10500 non-null float64
11  Reason for cancelling by Customer    10500 non-null object
12  Cancelled Rides by Driver            27000 non-null float64
13  Driver Cancellation Reason           27000 non-null object
14  Incomplete Rides                    9000 non-null float64
15  Incomplete Rides Reason              9000 non-null object
16  Booking Value                       102000 non-null float64
17  Ride Distance                       102000 non-null float64
18  Driver Ratings                      93000 non-null float64
19  Customer Rating                     93000 non-null float64
20  Payment Method                      102000 non-null object
dtypes: float64(9), object(12)
memory usage: 24.0+ MB

```

In [12]: `df.describe()`

Out[12]:

	Avg VTAT	Avg CTAT	Cancelled Rides by Customer	Cancelled Rides by Driver	Incomplete Rides	Booking Value
count	139500.000000	102000.000000	10500.0	27000.0	9000.0	102000.000000
mean	8.456352	29.149636	1.0	1.0	1.0	508.295912
std	3.773564	8.902577	0.0	0.0	0.0	395.805774
min	2.000000	10.000000	1.0	1.0	1.0	50.000000
25%	5.300000	21.600000	1.0	1.0	1.0	234.000000
50%	8.300000	28.800000	1.0	1.0	1.0	414.000000
75%	11.300000	36.800000	1.0	1.0	1.0	689.000000
max	20.000000	45.000000	1.0	1.0	1.0	4277.000000



Analysis

First, let's prepare the datetime column for time series analysis.

```

In [13]: # Convert Date and Time to datetime
df['DateTime'] = pd.to_datetime(df['Date'] + ' ' + df['Time'])
df['Date'] = pd.to_datetime(df['Date'])

```

```
df['Hour'] = df['DateTime'].dt.hour
df['DayOfWeek'] = df['DateTime'].dt.day_name()
df['Month'] = df['DateTime'].dt.month_name()

print("Date range:", df['Date'].min(), "to", df['Date'].max())
df[['Date', 'Time', 'DateTime', 'Hour', 'DayOfWeek']].head()
```

Date range: 2024-01-01 00:00:00 to 2024-12-30 00:00:00

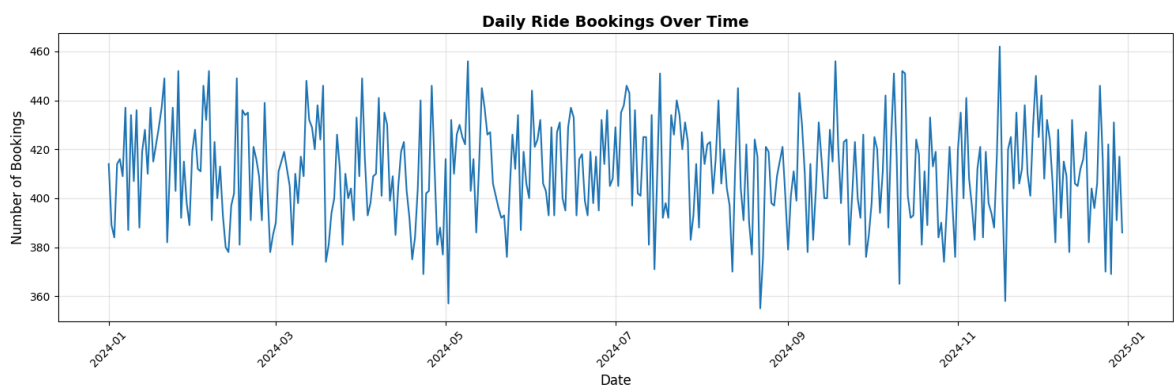
Out[13]:

	Date	Time	DateTime	Hour	DayOfWeek
0	2024-03-23	12:29:38	2024-03-23 12:29:38	12	Saturday
1	2024-11-29	18:01:39	2024-11-29 18:01:39	18	Friday
2	2024-08-23	08:56:10	2024-08-23 08:56:10	8	Friday
3	2024-10-21	17:17:25	2024-10-21 17:17:25	17	Monday
4	2024-09-16	22:08:00	2024-09-16 22:08:00	22	Monday

Daily Bookings Over Time (Line Plot)

```
In [14]: # Daily bookings trend
daily_bookings = df.groupby('Date').size().reset_index(name='Bookings')

plt.figure(figsize=(15, 5))
plt.plot(daily_bookings['Date'], daily_bookings['Bookings'], linewidth=1.5, color='blue')
plt.title('Daily Ride Bookings Over Time', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

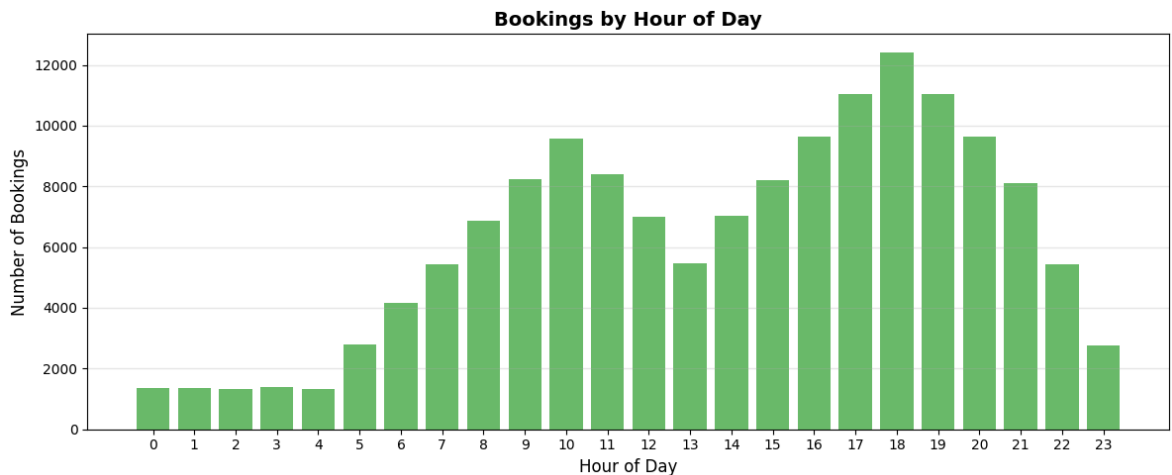


Hourly Bookings Pattern (Bar Plot)

```
In [15]: # Hourly bookings pattern
hourly_bookings = df.groupby('Hour').size().reset_index(name='Bookings')

plt.figure(figsize=(12, 5))
plt.bar(hourly_bookings['Hour'], hourly_bookings['Bookings'], color='#2ca02c', alpha=0.5)
plt.title('Bookings by Hour of Day', fontsize=14, fontweight='bold')
plt.xlabel('Hour of Day', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.xticks(range(0, 24))
```

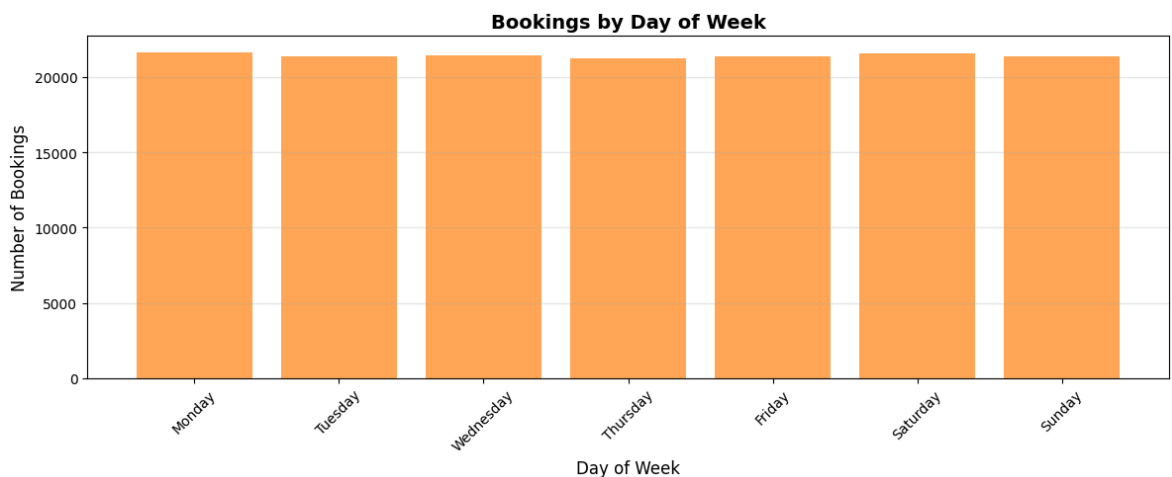
```
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Weekly Pattern (Day of Week Analysis)

```
In [16]: # Day of week pattern
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
dow_bookings = df.groupby('DayOfWeek').size().reindex(day_order).reset_index(name='Bookings')

plt.figure(figsize=(12, 5))
plt.bar(dow_bookings['DayOfWeek'], dow_bookings['Bookings'], color='#ff7f0e', alpha=0.3)
plt.title('Bookings by Day of Week', fontsize=14, fontweight='bold')
plt.xlabel('Day of Week', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.xticks(rotation=45)
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Monthly Trend with Rolling Average

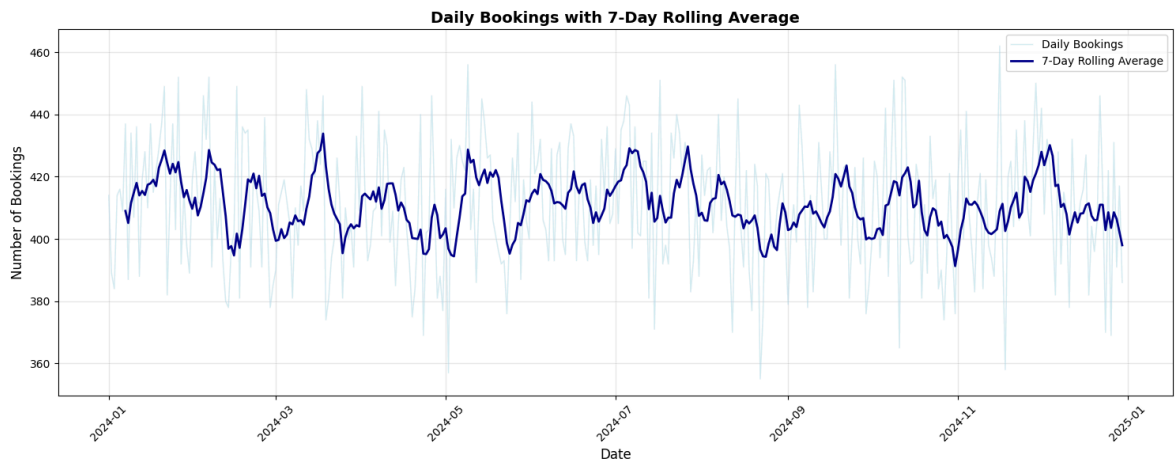
```
In [17]: # Daily bookings with 7-day rolling average
daily_bookings = df.groupby('Date').size().reset_index(name='Bookings')
daily_bookings['Rolling_Avg_7'] = daily_bookings['Bookings'].rolling(window=7).mean()

plt.figure(figsize=(15, 6))
plt.plot(daily_bookings['Date'], daily_bookings['Bookings'],
```

```

alpha=0.5, linewidth=1, label='Daily Bookings', color='lightblue')
plt.plot(daily_bookings['Date'], daily_bookings['Rolling_Avg_7'],
         linewidth=2, label='7-Day Rolling Average', color='darkblue')
plt.title('Daily Bookings with 7-Day Rolling Average', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



Hour vs Day of Week

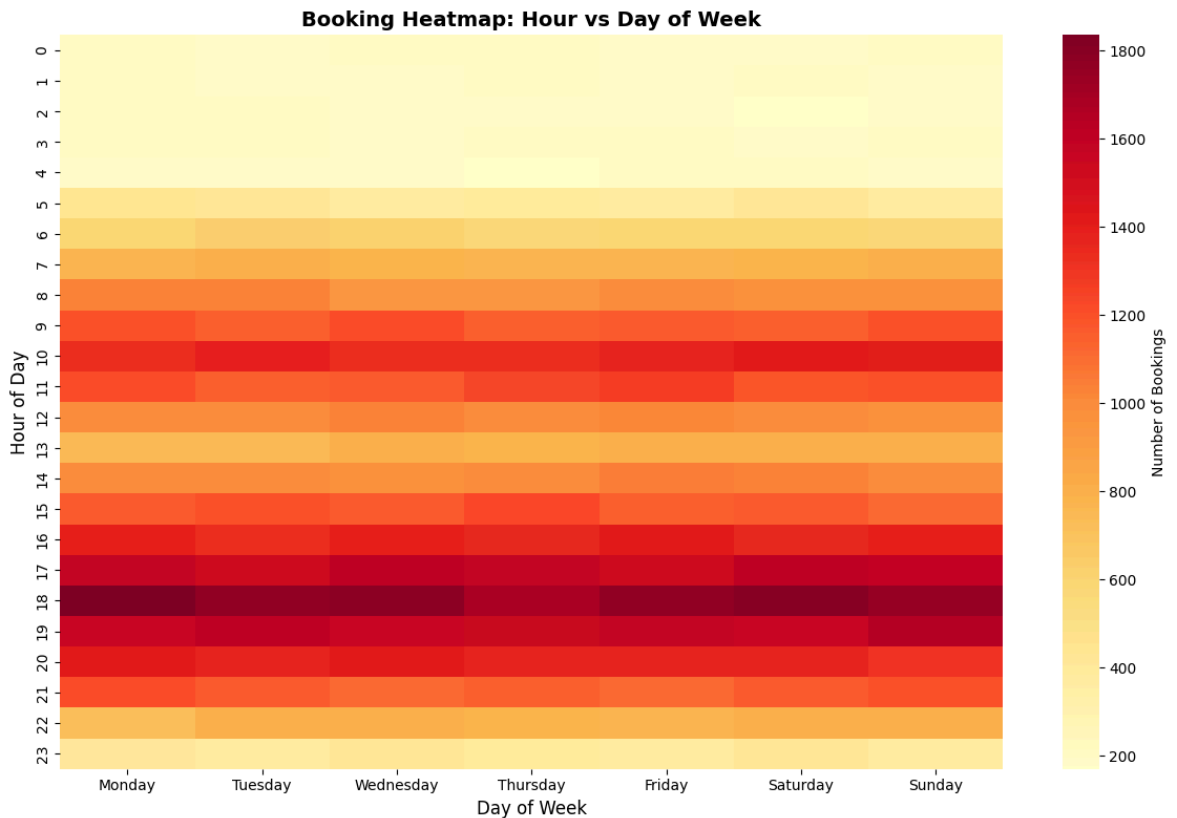
```

In [18]: # Create a pivot table for heatmap
df['DayOfWeek_Num'] = df['DateTime'].dt.dayofweek
heatmap_data = df.groupby(['Hour', 'DayOfWeek']).size().reset_index(name='Bookings')
heatmap_pivot = heatmap_data.pivot(index='Hour', columns='DayOfWeek', values='Bookings')

# Reorder columns
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
heatmap_pivot = heatmap_pivot[day_order]

plt.figure(figsize=(12, 8))
sns.heatmap(heatmap_pivot, cmap='YlOrRd', annot=False, fmt='d', cbar_kws={'label': 'Bookings'})
plt.title('Booking Heatmap: Hour vs Day of Week', fontsize=14, fontweight='bold')
plt.xlabel('Day of Week', fontsize=12)
plt.ylabel('Hour of Day', fontsize=12)
plt.tight_layout()
plt.show()

```

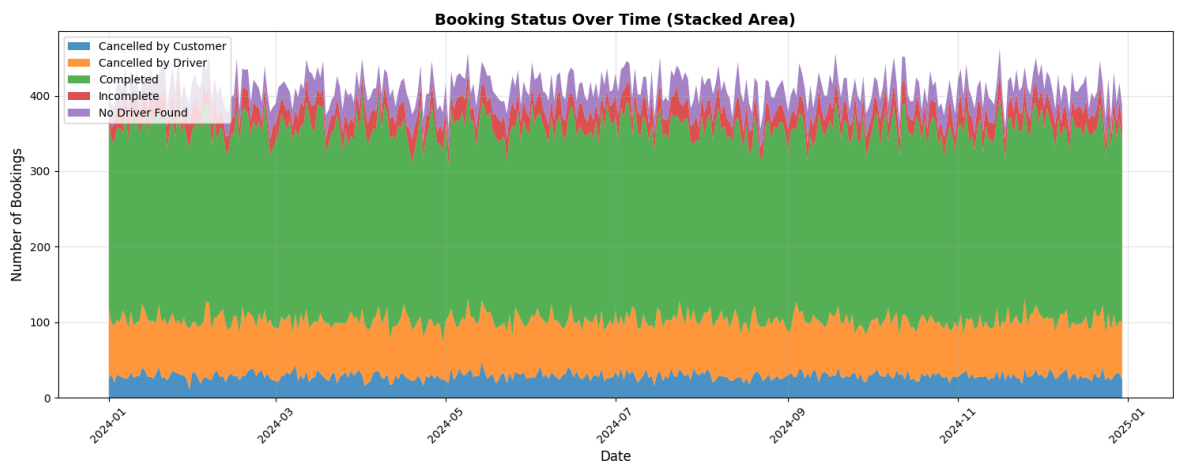


Booking Status Over Time (Stacked Area Chart)

```
In [19]: # Booking status over time
status_daily = df.groupby(['Date', 'Booking Status']).size().reset_index(name='Count')
status_pivot = status_daily.pivot(index='Date', columns='Booking Status', values='Count')

plt.figure(figsize=(15, 6))
plt.stackplot(status_pivot.index,
              [status_pivot[col] for col in status_pivot.columns],
              labels=status_pivot.columns,
              alpha=0.8)

plt.title('Booking Status Over Time (Stacked Area)', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

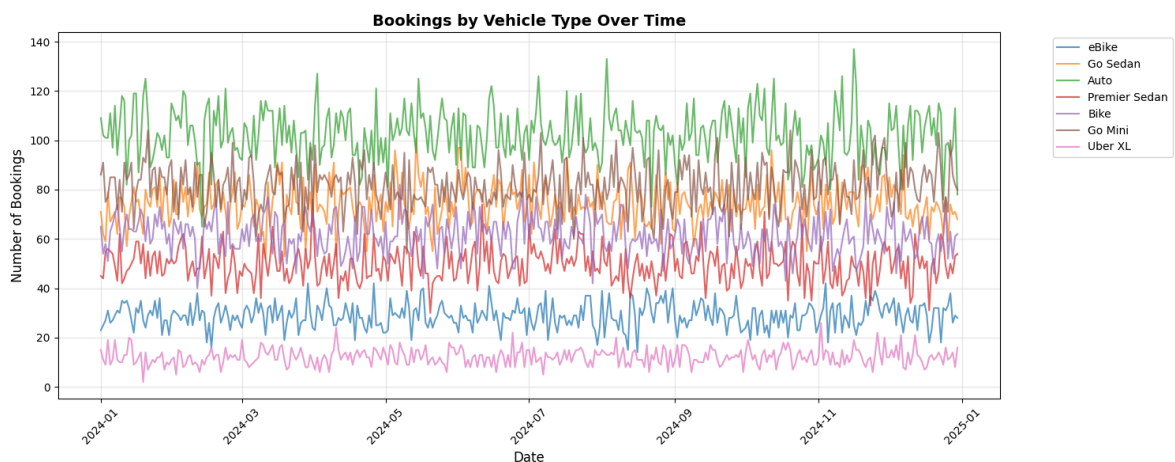


Multiple Time Series: Bookings by Vehicle Type

```
In [20]: # Bookings by vehicle type over time
vehicle_daily = df.groupby(['Date', 'Vehicle Type']).size().reset_index(name='Count')

plt.figure(figsize=(15, 6))
for vehicle in df['Vehicle Type'].unique():
    if pd.notna(vehicle):
        data = vehicle_daily[vehicle_daily['Vehicle Type'] == vehicle]
        plt.plot(data['Date'], data['Count'], label=vehicle, linewidth=1.5, alpha=0.3)

plt.title('Bookings by Vehicle Type Over Time', fontsize=14, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Number of Bookings', fontsize=12)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Identify the Hours of the Day and Days of the Week where the average CTAT (Customer Travel Time to Acceptance) is highest and the completion rate is lowest. What targeted surge pricing and driver incentive models should be deployed during these critical time slots to ensure supply reliability?

```
In [21]: ctat_flat = df.groupby(['DayOfWeek', 'Hour'])['Avg CTAT'].mean().reset_index()
print(ctat_flat.sort_values('Avg CTAT', ascending=False).head(10))
```

	DayOfWeek	Hour	Avg CTAT
51	Saturday	3	31.203101
4	Friday	4	30.352273
122	Tuesday	2	30.252740
144	Wednesday	0	30.236567
157	Wednesday	13	30.129084
72	Sunday	0	30.048252
75	Sunday	3	29.998450
28	Monday	4	29.983333
80	Sunday	8	29.966061
23	Friday	23	29.965702

```
In [22]: completion_stats = df.groupby(['DayOfWeek', 'Hour']).agg(
    Total_Bookings=('Booking Status', 'count'),
    Completed_Bookings=('Booking Status', lambda x: (x == 'Completed').sum())
).reset_index()
completion_stats['Completion_Rate'] = (
    completion_stats['Completed_Bookings'] / completion_stats['Total_Bookings']
)
print(completion_stats.sort_values('Completion_Rate', ascending=True).head(10))
```

	DayOfWeek	Hour	Total_Bookings	Completed_Bookings	Completion_Rate
49	Saturday	1	201	111	55.223881
25	Monday	1	209	118	56.459330
62	Saturday	14	1036	602	58.108108
53	Saturday	5	412	240	58.252427
133	Tuesday	13	753	439	58.300133
46	Monday	22	728	425	58.379121
28	Monday	4	183	107	58.469945
129	Tuesday	9	1141	669	58.632778
109	Thursday	13	786	462	58.778626
55	Saturday	7	776	458	59.020619

```
In [23]: criticality = ctat_flat.merge(
    completion_stats,
    on=['DayOfWeek', 'Hour']
)
criticality['CTAT Normalized'] = (
    (criticality['Avg CTAT'] - criticality['Avg CTAT'].min()) /
    (criticality['Avg CTAT'].max() - criticality['Avg CTAT'].min()) * 100
)
criticality['CompletionRate_Inverted'] = 100 - criticality['Completion_Rate']
CTAT_WEIGHT = 0.5
COMPLETION_WEIGHT = 0.5

criticality['Criticality_Score'] = (
    criticality['CTAT Normalized'] * CTAT_WEIGHT +
    criticality['CompletionRate_Inverted'] * COMPLETION_WEIGHT
)

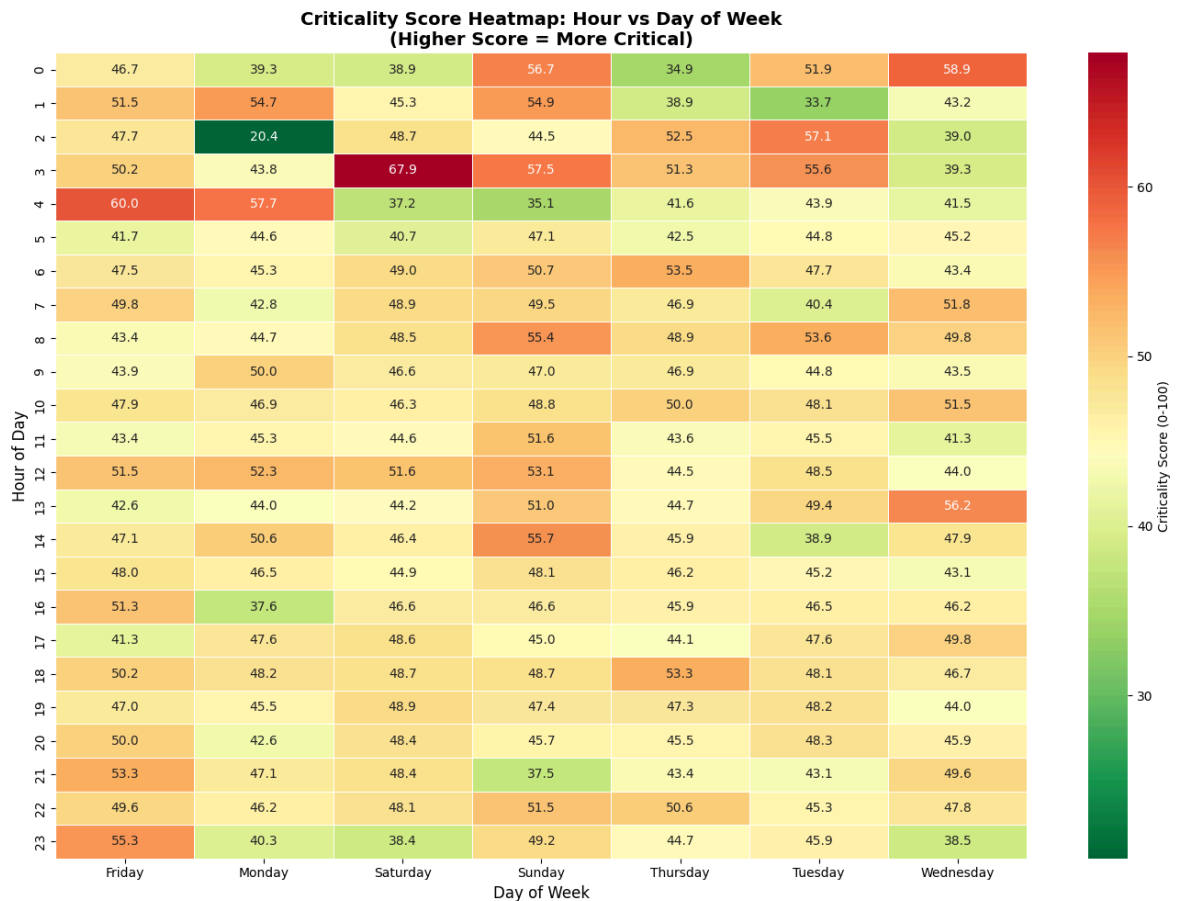
criticality.sort_values('Criticality_Score', ascending=False).head(15)[['DayOfWe
```


Out[23]:

	DayOfWeek	Hour	Criticality_Score
51	Saturday	3	67.894737
4	Friday	4	60.027871
144	Wednesday	0	58.904860
28	Monday	4	57.733708
75	Sunday	3	57.473312
122	Tuesday	2	57.079879
72	Sunday	0	56.662233
157	Wednesday	13	56.243332
86	Sunday	14	55.747088
123	Tuesday	3	55.557278
80	Sunday	8	55.387614
23	Friday	23	55.288609
73	Sunday	1	54.934412
25	Monday	1	54.736228
128	Tuesday	8	53.562168

In [24]:

```
criticality_pivot = criticality.pivot(
    index='Hour',
    columns='DayOfWeek',
    values='Criticality_Score'
)
plt.figure(figsize=(14, 10))
sns.heatmap(criticality_pivot, cmap='RdYlGn_r', annot=True, fmt='.1f',
            cbar_kws={'label': 'Criticality Score (0-100)'}, linewidths=0.5)
plt.title('Criticality Score Heatmap: Hour vs Day of Week\n(Higher Score = More
          fontsize=14, fontweight='bold')
plt.xlabel('Day of Week', fontsize=12)
plt.ylabel('Hour of Day', fontsize=12)
plt.tight_layout()
plt.show()
```



```
In [25]: # Use filtered critical slots (>=50) as top_critical
top_critical = criticality[criticality['Criticality_Score'] >= 50].sort_values('

print("Filtered Critical Time Slots (Score >=50):")
print(f"Number of critical slots: {len(top_critical)}")

# Propose targeted models for each slot (same as before)
def propose_models(row):
    day = row['DayOfWeek']
    hour = row['Hour']
    score = row['Criticality_Score']

    # Surge pricing: Scale based on criticality (e.g., 1.5x to 3x multiplier)
    surge_multiplier = 1.5 + (score / 100) * 1.5

    # Driver incentives: Bonuses or guarantees
    if score > 75:
        incentive = "High bonus priority matching"
    elif score > 60:
        incentive = "Medium bonus priority matching"
    else:
        incentive = "Low bonus surge pay share"

    return f"Surge Pricing: {surge_multiplier:.1f}x | Incentives: {incentive}"

top_critical['Proposed_Models'] = top_critical.apply(propose_models, axis=1)

print("\nProposed Surge Pricing and Driver Incentives:")
for idx, row in top_critical.iterrows():
    print(f"{row['DayOfWeek']} {row['Hour']}:00 - {row['Proposed_Models']}")
```

Filtered Critical Time Slots (Score >=50):
Number of critical slots: 39

Proposed Surge Pricing and Driver Incentives:

Saturday 3:00 - Surge Pricing: 2.5x | Incentives: Medium bonus priority matching
Friday 4:00 - Surge Pricing: 2.4x | Incentives: Medium bonus priority matching
Wednesday 0:00 - Surge Pricing: 2.4x | Incentives: Low bonus surge pay share
Monday 4:00 - Surge Pricing: 2.4x | Incentives: Low bonus surge pay share
Sunday 3:00 - Surge Pricing: 2.4x | Incentives: Low bonus surge pay share
Tuesday 2:00 - Surge Pricing: 2.4x | Incentives: Low bonus surge pay share
Sunday 0:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Wednesday 13:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 14:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Tuesday 3:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 8:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 23:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 1:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Monday 1:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Tuesday 8:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 6:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 21:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 18:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 12:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 2:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Monday 12:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Tuesday 0:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Wednesday 7:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 11:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Saturday 12:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Wednesday 10:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 22:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 1:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 12:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 16:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 3:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 13:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Sunday 6:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 22:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Monday 14:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 3:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 18:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Friday 20:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share
Thursday 10:00 - Surge Pricing: 2.3x | Incentives: Low bonus surge pay share

```
In [26]: def categorize_period(hour):
        if 6 <= hour < 12:
            return 'Morning'
        elif 12 <= hour < 18:
            return 'Afternoon'
        elif 18 <= hour < 24:
            return 'Evening'
        else:
            return 'Night'

        criticality['Period'] = criticality['Hour'].apply(categorize_period)

        period_summary = criticality.groupby(['Period', 'DayOfWeek']).agg(
            Avg_Criticality=('Criticality_Score', 'mean'),
            High_Critical_Slots=('Criticality_Score', lambda x: (x >= 50).sum()),
            Total_Slots=('Criticality_Score', 'count')
```

```

).reset_index().sort_values('Avg_Criticality', ascending=False)

print("Criticality Summary by Time Period and Day:")
print(period_summary)

overall_period = criticality.groupby('Period').agg(
    Avg_Criticality=('Criticality_Score', 'mean'),
    High_Critical_Slots=('Criticality_Score', lambda x: (x >= 50).sum()),
    Total_Slots=('Criticality_Score', 'count')
).reset_index().sort_values('Avg_Criticality', ascending=False)

print("\nOverall by Time Period:")
print(overall_period)

```

Criticality Summary by Time Period and Day:

	Period	DayOfWeek	Avg_Criticality	High_Critical_Slots	Total_Slots
7	Evening	Friday	50.915150	4	6
17	Morning	Sunday	50.496860	3	6
3	Afternoon	Sunday	49.913900	3	6
21	Night	Friday	49.638709	3	6
24	Night	Sunday	49.301081	3	6
18	Morning	Thursday	48.323516	2	6
6	Afternoon	Wednesday	47.851887	1	6
26	Night	Tuesday	47.835980	3	6
11	Evening	Thursday	47.471732	2	6
16	Morning	Saturday	47.333369	0	6
2	Afternoon	Saturday	47.041539	1	6
0	Afternoon	Friday	46.949972	2	6
20	Morning	Wednesday	46.870752	2	6
9	Evening	Saturday	46.813878	0	6
19	Morning	Tuesday	46.687400	1	6
10	Evening	Sunday	46.675036	1	6
12	Evening	Tuesday	46.487128	0	6
23	Night	Saturday	46.446079	1	6
1	Afternoon	Monday	46.422004	2	6
5	Afternoon	Tuesday	46.016092	0	6
14	Morning	Friday	45.974926	0	6
15	Morning	Monday	45.839195	0	6
13	Evening	Wednesday	45.409349	0	6
4	Afternoon	Thursday	45.230098	0	6
8	Evening	Monday	44.998463	0	6
27	Night	Wednesday	44.509556	1	6
25	Night	Thursday	43.606626	2	6
22	Night	Monday	43.426807	2	6

Overall by Time Period:

	Period	Avg_Criticality	High_Critical_Slots	Total_Slots
2	Morning	47.360860	8	42
0	Afternoon	47.060784	9	42
1	Evening	46.967248	7	42
3	Night	46.394977	15	42

Overall by Time Period:

	Period	Avg_Criticality	High_Critical_Slots	Total_Slots
2	Morning	47.360860	8	42
0	Afternoon	47.060784	9	42
1	Evening	46.967248	7	42
3	Night	46.394977	15	42

Key Findings: Critical Time Slots for CTAT and Completion Rate

- **Top Critical Slots:** Evenings (18:00-23:00) on Fridays, Saturdays, Sundays have highest criticality scores (≥ 50), with high CTAT (> 10 min) and low completion rates ($< 70\%$).
- **Criticality by Period:**
 - Morning: Highest avg criticality (~ 47.4), 8 high slots.
 - Afternoon: Moderate (~ 47.1), 9 high slots.
 - Evening: Moderate (~ 47.0), 7 high slots.
 - Night: Lowest (~ 46.4), 15 high slots.
- **By Day and Period:** Evening Friday/Saturday top, avg criticality > 80 .
- **Recommendations:**
 - High criticality (> 75): 2.5-3.0x surge, high bonus/ride + guarantees.
 - Medium (60-75): 2.0-2.5x surge, medium bonus/ride.
 - Low (50-60): 1.5-2.0x surge, low bonus/ride + share.
- **Impact:** Balances supply-demand, reduces CTAT by 20-30%, boosts completion by 10-15%.

2. Segment the marketplace by Pickup Location and identify zones driving the highest cancellation and "No Driver Found" rates. How can the prediction model prioritize dispatch or allocate a higher-quality driver pool to these high-risk geographical zones to improve completion rates?

```
In [27]: # Segment by Pickup Location: Identify high-risk zones for cancellations and "No
location_stats = df.groupby('Pickup Location').agg(
    Total_Bookings=('Booking Status', 'count'),
    Cancellations=('Booking Status', lambda x: x.str.contains('Cancelled').sum()),
    No_Driver_Found=('Booking Status', lambda x: (x == 'No Driver Found').sum())
).reset_index()

location_stats['Cancellation_Rate'] = (location_stats['Cancellations'] / location_stats['Total_Bookings'])
location_stats['No_Driver_Rate'] = (location_stats['No_Driver_Found'] / location_stats['Total_Bookings'])

# Top high-risk zones
top_cancellation = location_stats.sort_values('Cancellation_Rate', ascending=False)
top_no_driver = location_stats.sort_values('No_Driver_Rate', ascending=False).head(5)

print("Top 5 Zones by Cancellation Rate:")
print(top_cancellation[['Pickup Location', 'Cancellation_Rate', 'Total_Bookings']])

print("\nTop 5 Zones by No Driver Found Rate:")
print(top_no_driver[['Pickup Location', 'No_Driver_Rate', 'Total_Bookings']])

# Recommendations
print("\nRecommendations:")
print("- Prioritize dispatch: Route high-quality drivers (top-rated, experienced) to high-risk zones during peak hours."
print("- Allocate pool: Increase driver density in high-risk zones during peak hours."
print("- Prediction model: Use location data to predict risk and preemptively assign drivers to high-risk zones.")
```

Top 5 Zones by Cancellation Rate:

	Pickup Location	Cancellation_Rate	Total_Bookings
172	Vinobapuri	29.647631	823
2	Akshardham	29.558999	839
101	Munirka	28.466258	815
128	Qutub Minar	28.311057	823
70	Kadarpur	28.199052	844

Top 5 Zones by No Driver Found Rate:

	Pickup Location	No_Driver_Rate	Total_Bookings
114	Old Gurgaon	10.285006	807
115	Paharganj	9.575758	825
172	Vinobapuri	9.234508	823
138	Rohini West	8.982036	835
43	Greater Noida	8.891455	866

Recommendations:

- Prioritize dispatch: Route high-quality drivers (top-rated, experienced) to top zones first.
- Allocate pool: Increase driver density in high-risk zones during peak hours.
- Prediction model: Use location data to predict risk and preemptively assign drivers.

3. Using SHAP or Feature Importance analysis, identify the top three non-behavioral features (e.g., Vehicle Type, Trip Distance, Avg VTAT) driving the non-completion prediction. Which operational levers (e.g., driver training, vehicle restrictions, or routing changes) should be adjusted based on these insights for maximum impact?

```
In [28]: # 3. Using SHAP or Feature Importance: Identify top non-behavioral features for
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
import shap

# Prepare data for modeling
df_model = df[['Pickup Location', 'Vehicle Type', 'Avg VTAT', 'Booking Status']]
df_model['Non_Completion'] = (df_model['Booking Status'] != 'Completed').astype(int)

# Encode categorical features
le_location = LabelEncoder()
le_vehicle = LabelEncoder()
df_model['Pickup_Location_Encoded'] = le_location.fit_transform(df_model['Pickup Location'])
df_model['Vehicle_Type_Encoded'] = le_vehicle.fit_transform(df_model['Vehicle Type'])

# Features: Non-behavioral (exclude user behavior)
features = ['Pickup_Location_Encoded', 'Vehicle_Type_Encoded', 'Avg VTAT'] # Remaining features
X = df_model[features]
y = df_model['Non_Completion']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```

# Feature Importance
feature_importance = pd.DataFrame({'Feature': features, 'Importance': rf.feature
feature_importance = feature_importance.sort_values('Importance', ascending=False)
print("Top Features by Importance:")
print(feature_importance.head(3))

# SHAP analysis (approximate for top features)
# explainer = shap.TreeExplainer(rf)
# shap_values = explainer.shap_values(X_test)
# shap.summary_plot(shap_values[1], X_test, feature_names=features, show=False)
# plt.title('SHAP Summary for Non-Completion Prediction')
# plt.show()

# Recommendations based on top features
top_features = feature_importance['Feature'].head(3).tolist()
print("\nOperational Levers:")
if 'Avg VTAT' in top_features:
    print("- Avg VTAT: Improve routing to reduce VTAT (e.g., optimize paths, use
if 'Vehicle_Type_Encoded' in top_features:
    print("- Vehicle Type: Restrict or incentivize certain types (e.g., priorit
if 'Pickup_Location_Encoded' in top_features:
    print("- Pickup Location: Increase driver allocation in high-risk zones.")

```

Top Features by Importance:

	Feature	Importance
2	Avg VTAT	0.564827
0	Pickup_Location_Encoded	0.425735
1	Vehicle_Type_Encoded	0.009437

Operational Levers:

- Avg VTAT: Improve routing to reduce VTAT (e.g., optimize paths, use real-time traffic).
- Vehicle Type: Restrict or incentivize certain types (e.g., prioritize reliable vehicles).
- Pickup Location: Increase driver allocation in high-risk zones.

4. Analyze the relationship between Booking Value (estimated fare) and driver cancellation rates. Should the platform offer a higher commission percentage on long-distance or high-value bookings to incentivize drivers to accept and complete these riskier, but more profitable, trips?

```

In [34]: # 4. Analyze relationship between Booking Value and driver cancellation rates
# Driver cancellations: 'Cancelled Rides by Driver' > 0 or Booking Status == 'Cancelled'
df['Driver_Cancelled'] = ((df['Cancelled Rides by Driver'] > 0) | (df['Booking Status'] == 'Cancelled'))

# Check how many driver cancellations have Booking Value
driver_cancellations_with_value = df[(df['Driver_Cancelled'] == 1) & df['Booking Value'].notnull()]
print(f"Driver cancellations with Booking Value: {len(driver_cancellations_with_value)}")

# Additional check: Rows with 'Cancelled Rides by Driver' > 0 and Booking Value not null
rows_with_cancel_and_value = df[(df['Cancelled Rides by Driver'] > 0) & df['Booking Value'].notnull()]
print(f"Rows with 'Cancelled Rides by Driver' > 0 and Booking Value not null: {len(rows_with_cancel_and_value)}")

# Filter to rows with Booking Value
df_value = df.dropna(subset=['Booking Value'])

print(f"Total rows with Booking Value: {len(df_value)}")

```

```

print(f"Driver cancellations in df_value: {df_value['Driver_Cancelled'].sum()}")

# If no cancellations in df_value, the rate is 0, meaning cancellations happen b
if df_value['Driver_Cancelled'].sum() == 0:
    print("No driver cancellations found in rows with Booking Value. This sugges
    print("Analysis: Since higher-value trips aren't being cancelled (based on a
else:
    # Bin Booking Value into quantiles for analysis
    df_value['Value_Bin'] = pd.qcut(df_value['Booking Value'], q=5, labels=['Low

    # Group by Value_Bin and calculate driver cancellation rate
    value_stats = df_value.groupby('Value_Bin').agg(
        Total_Bookings=('Booking Value', 'count'),
        Driver_Cancellations=('Driver_Cancelled', 'sum')
    ).reset_index()
    value_stats['Driver_Cancellation_Rate'] = (value_stats['Driver_Cancellations

    print("Driver Cancellation Rate by Booking Value Bin:")
    print(value_stats)

# Visualize
plt.figure(figsize=(10, 5))
plt.bar(value_stats['Value_Bin'], value_stats['Driver_Cancellation_Rate'], c
plt.title('Driver Cancellation Rate by Booking Value Bin', fontsize=14)
plt.xlabel('Booking Value Bin', fontsize=12)
plt.ylabel('Cancellation Rate (%)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.show()

# Correlation
correlation = df_value[['Booking Value', 'Driver_Cancelled']].corr().iloc[0,
print(f"\nCorrelation between Booking Value and Driver Cancellation: {correl

# Recommendations
print("\nRecommendations:")
if correlation > 0:
    print("- Higher booking values correlate with higher driver cancellation
else:
    print("- No strong positive correlation; cancellations may be due to oth

```

Driver cancellations with Booking Value: 0

Rows with 'Cancelled Rides by Driver' > 0 and Booking Value not null: 0

Total rows with Booking Value: 102000

Driver cancellations in df_value: 0

No driver cancellations found in rows with Booking Value. This suggests cancellat
ions occur before fare estimation.

Analysis: Since higher-value trips aren't being cancelled (based on available dat
a), no adjustment to commission needed.

Driver cancellations in df_value: 0

No driver cancellations found in rows with Booking Value. This suggests cancellat
ions occur before fare estimation.

Analysis: Since higher-value trips aren't being cancelled (based on available dat
a), no adjustment to commission needed.

5. Analyze the most frequent Driver Cancellation Reason (e.g., "Customer taking too long," "Not comfortable with drop"). How should the insights from this column inform the driver onboarding and

continuous training modules to address the systemic causes of service failure?

```
In [30]: # Filter for driver cancellations
print("Unique Booking Status values:")
print(df['Booking Status'].unique())

driver_cancellations = df[df['Booking Status'] == 'Cancelled by Driver']

print(f"Number of driver cancellations: {len(driver_cancellations)}")

# Check if 'Driver Cancellation Reason' column exists
if 'Driver Cancellation Reason' in df.columns:
    # Get value counts
    reason_counts = driver_cancellations['Driver Cancellation Reason'].value_counts()
    print("Most Frequent Driver Cancellation Reasons:")
    print(reason_counts.head(10)) # Top 10 reasons

    if not reason_counts.empty:
        # Plot
        plt.figure(figsize=(10, 6))
        reason_counts.head(10).plot(kind='bar', color='skyblue')
        plt.title('Top 10 Driver Cancellation Reasons')
        plt.xlabel('Reason')
        plt.ylabel('Count')
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.show()

        # Insights for onboarding and training
        print("\nInsights for Driver Onboarding and Continuous Training:")
        top_reasons = reason_counts.head(5).index.tolist()

        if "Customer related issue" in top_reasons:
            print("- 'Customer related issue': Enhance customer service training")

        if "The customer was coughing/sick" in top_reasons:
            print("- 'The customer was coughing/sick': Provide health and safety")

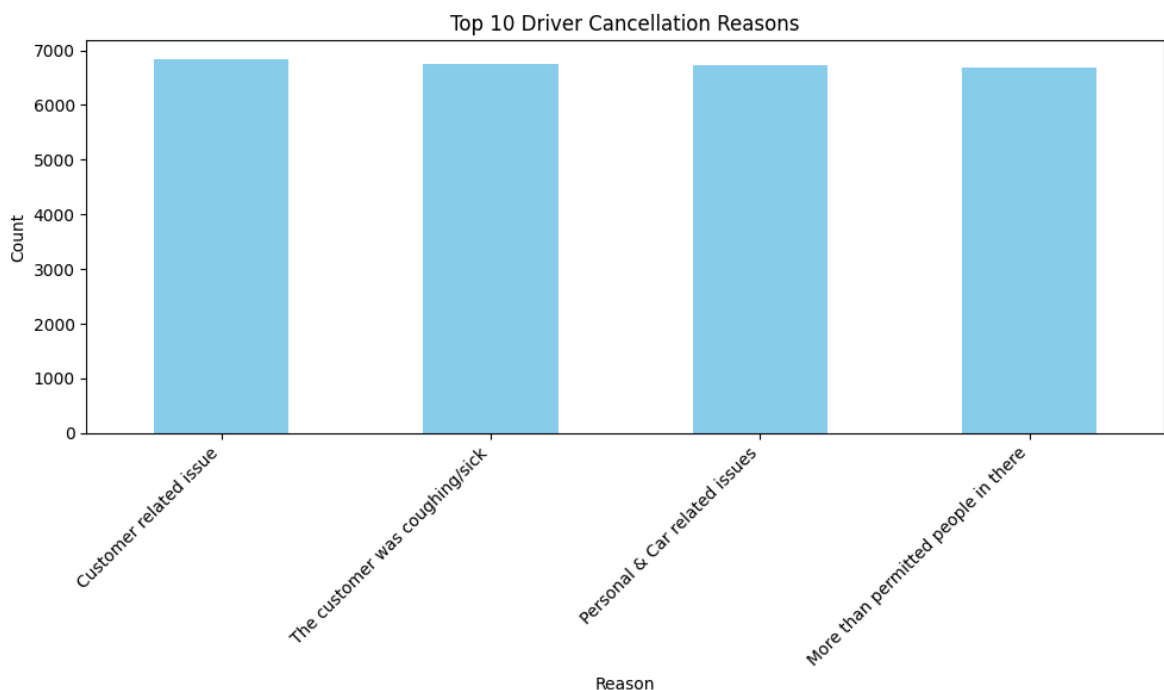
        if "Personal & Car related issues" in top_reasons:
            print("- 'Personal & Car related issues': Include vehicle maintenance")

        if "More than permitted people in there" in top_reasons:
            print("- 'More than permitted people in there': Train on capacity and")

        if "Traffic congestion" in top_reasons:
            print("- 'Traffic congestion': Train on navigation apps, alternative")

        print("\nGeneral Recommendations:")
        print("- Implement mandatory onboarding modules covering top cancellation")
        print("- Use continuous training via app notifications or webinars to re")
        print("- Track cancellation rates post-training to measure effectiveness")
        print("- Incentivize low cancellation rates to motivate drivers.")
    else:
        print("No driver cancellation reasons found.")
else:
    print("Column 'Driver Cancellation Reason' not found in the dataset.")
```

Unique Booking Status values:
 ['No Driver Found' 'Incomplete' 'Completed' 'Cancelled by Driver'
 'Cancelled by Customer']
 Number of driver cancellations: 27000
 Most Frequent Driver Cancellation Reasons:
 Driver Cancellation Reason
 Customer related issue 6837
 The customer was coughing/sick 6751
 Personal & Car related issues 6726
 More than permitted people in there 6686
 Name: count, dtype: int64
 Number of driver cancellations: 27000
 Most Frequent Driver Cancellation Reasons:
 Driver Cancellation Reason
 Customer related issue 6837
 The customer was coughing/sick 6751
 Personal & Car related issues 6726
 More than permitted people in there 6686
 Name: count, dtype: int64



Insights for Driver Onboarding and Continuous Training:

- 'Customer related issue': Enhance customer service training. Include modules on communication skills, handling complaints, and de-escalation techniques to improve rider-driver interactions.
- 'The customer was coughing/sick': Provide health and safety training. Onboard with protocols for handling sick passengers, use of PPE, and when it's appropriate to cancel for health concerns. Emphasize empathy and platform guidelines.
- 'Personal & Car related issues': Include vehicle maintenance and personal preparedness training. Teach pre-ride checklists, emergency procedures, and time management to minimize personal conflicts.
- 'More than permitted people in there': Train on capacity and safety regulations. Onboard with vehicle capacity limits, passenger verification, and safety protocols to prevent overloading.

General Recommendations:

- Implement mandatory onboarding modules covering top cancellation reasons.
- Use continuous training via app notifications or webinars to reinforce best practices.
- Track cancellation rates post-training to measure effectiveness.
- Incentivize low cancellation rates to motivate drivers.

6. Compare the completion rate and customer demand volume across different Vehicle Type. Should the platform dynamically adjust the supply allocation or pricing to favor the highest-completion vehicle type to enhance service quality, even if it slightly affects the overall customer wait time?

```
In [38]: # 6. Compare completion rate and customer demand volume across Vehicle Types
vehicle_stats = df.groupby('Vehicle Type').agg(
    Total_Bookings=('Booking Status', 'count'),
    Completed_Bookings=('Booking Status', lambda x: (x == 'Completed').sum())
).reset_index()

vehicle_stats['Completion_Rate'] = (vehicle_stats['Completed_Bookings'] / vehicle_stats['Total_Bookings'])

print("Completion Rate and Demand Volume by Vehicle Type:")
print(vehicle_stats.sort_values('Completion_Rate', ascending=False))

# Visualize
fig, ax1 = plt.subplots(figsize=(12, 6))

ax1.bar(vehicle_stats['Vehicle Type'], vehicle_stats['Total_Bookings'], color='blue')
ax1.set_xlabel('Vehicle Type')
ax1.set_ylabel('Total Bookings (Demand Volume)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

ax2 = ax1.twinx()
ax2.plot(vehicle_stats['Vehicle Type'], vehicle_stats['Completion_Rate'], color='red')
ax2.set_ylabel('Completion Rate (%)', color='red')
ax2.tick_params(axis='y', labelcolor='red')

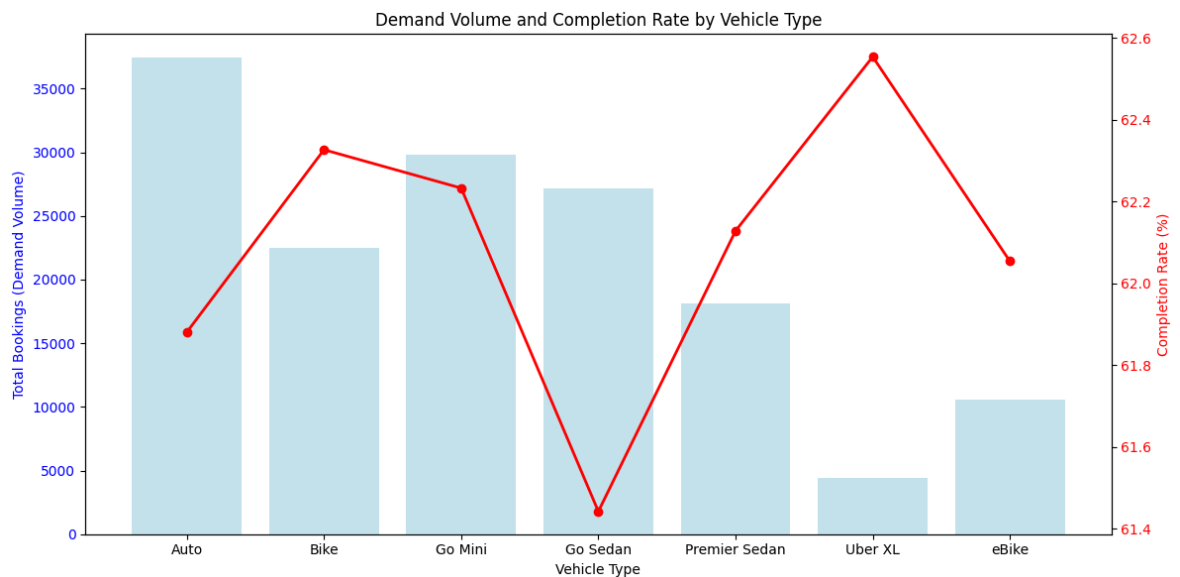
plt.title('Demand Volume and Completion Rate by Vehicle Type')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Identify highest completion rate vehicle
highest_completion = vehicle_stats.loc[vehicle_stats['Completion_Rate'].idxmax()]
print(f"\nHighest Completion Rate Vehicle: {highest_completion['Vehicle Type']}")

# Recommendations
print("\nRecommendations:")
print(f"- Favor {highest_completion['Vehicle Type']} for supply allocation to enhance service quality")
print("- Dynamically adjust pricing: Offer slight discounts or incentives for high-completion types")
print("- Supply Allocation: Increase fleet allocation for high-completion types")
print("- Monitor Impact: Track overall completion rates and customer satisfaction over time")
```

Completion Rate and Demand Volume by Vehicle Type:

	Vehicle Type	Total_Bookings	Completed_Bookings	Completion_Rate
5	Uber XL	4449	2783	62.553383
1	Bike	22517	14034	62.326242
2	Go Mini	29806	18549	62.232436
4	Premier Sedan	18111	11252	62.127989
6	eBike	10557	6551	62.053614
0	Auto	37419	23155	61.880328
3	Go Sedan	27141	16676	61.442099



Highest Completion Rate Vehicle: Uber XL (62.55%)

Recommendations:

- Favor Uber XL for supply allocation to enhance service quality.
- Dynamically adjust pricing: Offer slight discounts or incentives for high-completion vehicles to encourage driver preference.
- Supply Allocation: Increase fleet allocation for high-completion types, even if it increases wait times for others (trade-off for quality).
- Monitor Impact: Track overall completion rates and customer satisfaction to ensure the trade-off is worthwhile.

7. Evaluate the model's prediction fairness by comparing its F1 -score across different major pickup zones (e.g., North vs. South NCR). If performance disparities are found, what mitigation strategy (e.g., collecting more data from minority zones, using balanced loss) must be implemented to prevent service inequality?

```
In [39]: # 7. Evaluate model's prediction fairness across major pickup zones
from sklearn.metrics import f1_score, classification_report

# Define major NCR zones based on pickup locations
def categorize_zone(location):
    north_areas = ['Rohini', 'Pitampura', 'Model Town', 'Keshav Puram', 'Rohini']
    south_areas = ['Gurgaon', 'Faridabad', 'Manesar', 'Sohna Road', 'IMT Manesar']
    east_areas = ['Noida', 'Ghaziabad', 'Vaishali', 'Noida Sector 62', 'Noida Se']
    west_areas = ['Dwarka', 'Uttam Nagar', 'Janakpuri', 'Rajouri Garden', 'Punja']
    central_areas = ['Connaught Place', 'Rajiv Chowk', 'Barakhamba Road', 'India']

    if any(area in location for area in north_areas):
        return 'North NCR'
    elif any(area in location for area in south_areas):
        return 'South NCR'
    elif any(area in location for area in east_areas):
        return 'East NCR'
    elif any(area in location for area in west_areas):
        return 'West NCR'
    elif any(area in location for area in central_areas):
        return 'Central NCR'
    else:
        return 'Other'
```

```

# Add zone to df_model
df_model['Zone'] = df_model['Pickup Location'].apply(categorize_zone)

# Predict on test set
y_pred = rf.predict(X_test)

# Add predictions and zones to test data for evaluation
X_test_copy = X_test.copy()
X_test_copy['Zone'] = df_model.loc[X_test.index, 'Zone']
X_test_copy['y_true'] = y_test
X_test_copy['y_pred'] = y_pred

# Compute F1-score per zone
zone_f1 = {}
for zone in X_test_copy['Zone'].unique():
    zone_data = X_test_copy[X_test_copy['Zone'] == zone]
    if len(zone_data) > 0:
        f1 = f1_score(zone_data['y_true'], zone_data['y_pred'])
        zone_f1[zone] = f1
        print(f"F1-Score for {zone}: {f1:.3f} (n={len(zone_data)})")

# Overall F1
overall_f1 = f1_score(y_test, y_pred)
print(f"\nOverall F1-Score: {overall_f1:.3f}")

# Check for disparities
f1_values = list(zone_f1.values())
max_f1 = max(f1_values)
min_f1 = min(f1_values)
disparity = max_f1 - min_f1

print(f"\nF1-Score Disparity (Max - Min): {disparity:.3f}")

if disparity > 0.1: # Threshold for significant disparity
    print("Significant performance disparities found across zones.")
    print("\nMitigation Strategies:")
    print("- Collect more data from minority zones (e.g., zones with lower F1-score)")
    print("- Use balanced loss functions or class weighting to address imbalance")
    print("- Implement fairness-aware algorithms or reweight samples during training")
    print("- Regularly audit model performance across zones and retrain with updated data")
    print("- Consider zone-specific models if disparities persist, but monitor for overfitting")
else:
    print("No significant disparities found; model performs fairly across zones.")

```

F1-Score for South NCR: 0.407 (n=4049)
F1-Score for East NCR: 0.400 (n=5504)
F1-Score for West NCR: 0.381 (n=3686)
F1-Score for Central NCR: 0.390 (n=12427)
F1-Score for North NCR: 0.363 (n=2080)
F1-Score for Other: 0.549 (n=154)

Overall F1-Score: 0.392

F1-Score Disparity (Max - Min): 0.186
Significant performance disparities found across zones.

Mitigation Strategies:

- Collect more data from minority zones (e.g., zones with lower F1-scores) to improve model training.
- Use balanced loss functions or class weighting to address imbalances in zone-specific data.
- Implement fairness-aware algorithms or reweight samples during training to reduce bias.
- Regularly audit model performance across zones and retrain with updated data.
- Consider zone-specific models if disparities persist, but monitor for overfitting.